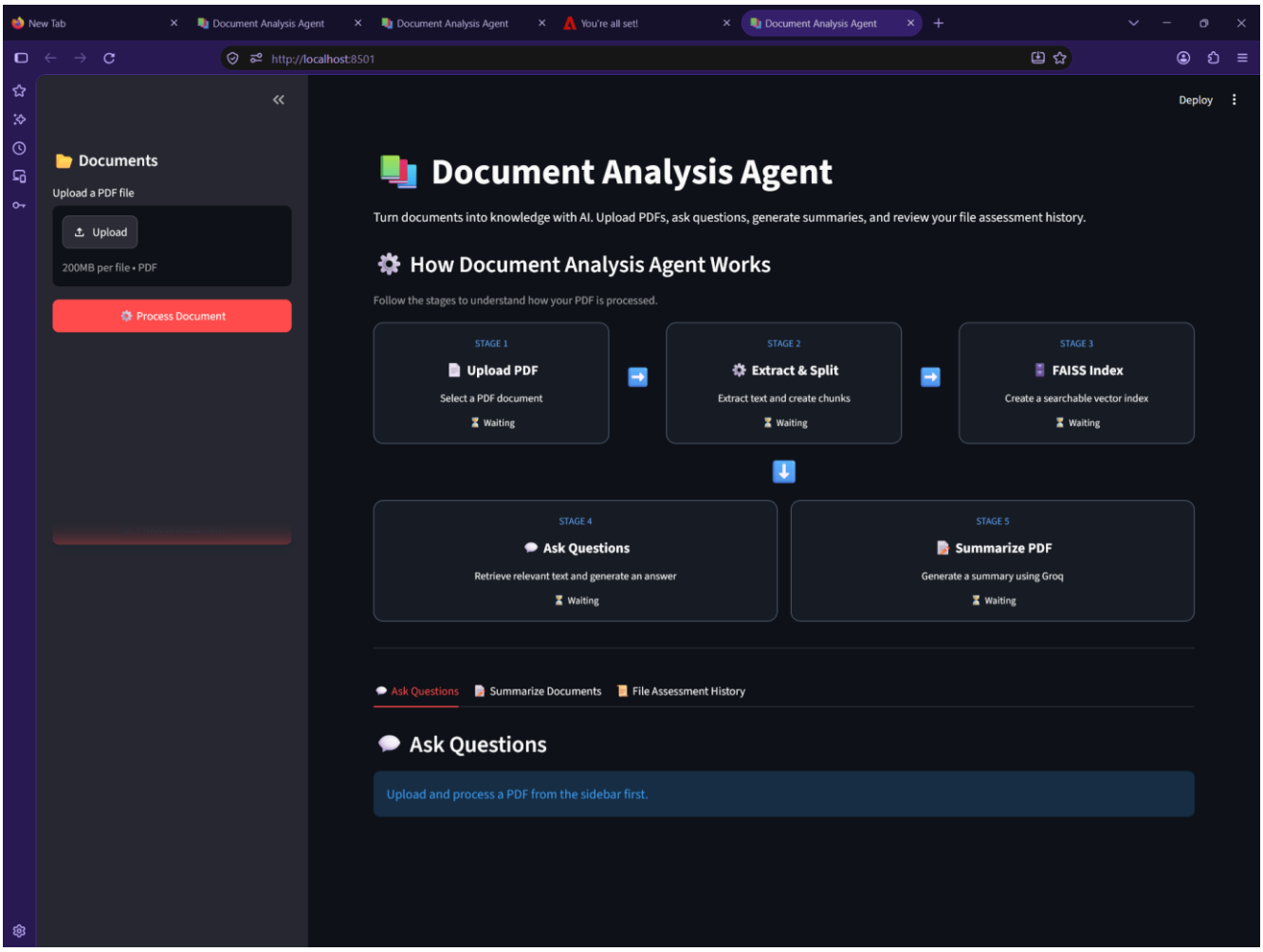
PROJECT DOCUMENTATION

Document Analysis Agent

An AI agent for PDF question answering, summarization, and document assessment history

Project type	AI Agent for PDF document analysis
Frontend	Streamlit
Backend API	FastAPI
AI / Retrieval	Groq ChatGroq + Sentence Transformers + FAISS

Prepared from the supplied frontend and backend source files



1. Project Overview

Document Analysis Agent is a web application for extracting knowledge from PDF documents. The user uploads a PDF through a Streamlit interface. A FastAPI backend extracts readable text, splits it into chunks, converts the chunks into embeddings, and stores them in a FAISS vector index. Users can then ask questions about the active PDF and generate a summary.

The frontend also records a session-based file assessment history containing processed-file details, tasks performed, questions, answers, summaries, timestamps, and task status. The supplied implementation keeps the active vector index in backend memory and the activity history in Streamlit session state.

2. Objectives

- Allow users to upload and process a PDF file.
- Extract text from PDF pages and retain page/file metadata.
- Split text into overlapping chunks suitable for retrieval.
- Create semantic embeddings and a FAISS similarity-search index.
- Answer questions using relevant document passages (RAG).
- Generate a readable summary of the active document.
- Show source file and page information for question answers.
- Track document processing and analysis activities in the current Streamlit session.
- Reset the active document without clearing the activity history.

3. Main Features

Feature	Description
PDF upload and processing	Accepts one PDF in the Streamlit sidebar and sends it to POST /upload.
Text extraction	Uses pypdf to read page text. Empty, invalid, or text-unreadable PDFs return an error; scanned PDFs may need OCR.
Chunking	Uses RecursiveCharacterTextSplitter with chunk_size=1000 and chunk_overlap=150 for retrieval chunks.
Vector search	Uses sentence-transformers/all-MiniLM-L6-v2 embeddings with FAISS.
Question answering	POST /ask retrieves up to four similar chunks and asks the Groq model to answer only from the provided context.
Source references	The answer response includes source file names and page numbers for retrieved chunks.
PDF summarization	POST /summarize summarizes text in larger chunks (6000 characters, 200 overlap), then combines section summaries when necessary.
Activity history	Records processing, questions, answers, summaries, timestamps, and completion status in Streamlit session state.
Document status	GET /status reports whether a document is loaded and provides filename, page count, and chunk count.
Reset	POST /reset clears the active backend index and document metadata; the frontend retains its history.
API documentation	FastAPI provides interactive API documentation at /docs when the backend is running.

4. Technology Stack

Technology	Role in this project
Python	Main programming language.
Streamlit	User interface, tabs, sidebar, metrics, workflow diagram, and session-state history.
FastAPI	REST API endpoints for upload, status, question answering, summarization, and reset.
Uvicorn	ASGI server used to run the FastAPI application.
pypdf	Reads PDF pages and extracts text.
LangChain	Document objects, text splitting, embeddings/vector-store integration, and Groq chat model integration.
ChatGroq	Calls the Groq-hosted model configured as openai/gpt-oss-120b.
Sentence Transformers	Creates semantic embeddings using all-MiniLM-L6-v2.
FAISS	Stores vectors and retrieves semantically similar text chunks.
python-dotenv	Loads GROQ_API_KEY from an environment file.
Requests	Streamlit frontend communicates with the FastAPI backend over HTTP.

5. System Architecture

User → Streamlit UI → FastAPI REST API → PDF text extraction → Text chunks → Embeddings → FAISS index

For question answering, the question is embedded and matched against the FAISS index. The most relevant chunks are assembled into a context prompt, which is sent to ChatGroq. The backend returns the answer and the file/page metadata for the retrieved passages. For summarization, the extracted chunks are combined, split into larger text sections, summarized by the language model, and merged into a final summary when needed.

6. Project File Structure

```
DOCUMENT_ANALYSIS_GROQ/
├── app.py              # Streamlit frontend (the supplied file is named main(1).py)
├── api.py              # FastAPI backend
├── requirements.txt    # Python dependencies
├── .env                # GROQ_API_KEY (keep private; do not commit)
├── .gitignore          # Ignore secrets, virtual environments, and caches
└── README.md           # Setup and usage instructions
```

The supplied frontend currently sets API_URL to http://127.0.0.1:8000. The backend resolves the environment file using Path(__file__).resolve().parent.parent / '.env'. Ensure this matches your actual folder layout; if .env is inside the same directory as api.py, adjust the path or load it from the project directory.

7. Endpoints

Method	Path	Purpose
GET	/	Health-style welcome message and endpoint list.
GET	/status	Returns loaded state, filename, page count, and chunk count.
POST	/upload	Accepts multipart form data with a PDF field named file; extracts and indexes text.

POST	/ask	Accepts JSON such as {"question": "What is the main topic?"}; returns answer and sources.
POST	/summarize	Generates a summary for the active PDF; no request body required.
POST	/reset	Clears the active in-memory document and vector index.
GET	/docs	Interactive Swagger API documentation generated by FastAPI.

8. How the Application Works

1. Start the FastAPI backend with Uvicorn.
2. Start the Streamlit frontend in a separate terminal.
3. Choose a PDF in the sidebar and click Process Document.
4. The frontend sends the file to POST /upload.
5. The backend extracts readable text from each page and stores page/file metadata.
6. Text is split into overlapping chunks; embeddings are generated and indexed in FAISS.
7. In Ask Questions, enter a question. The backend retrieves relevant chunks and generates an answer grounded in those passages.
8. In Summarize Documents, click Generate Summary to summarize the active PDF.
9. Review the File Assessment History tab to see recorded actions in the current session.
10. Use Reset Current Document to clear the active document while keeping the frontend history.

9. Setup and Run Instructions

9.1 Create and activate a virtual environment (Windows PowerShell)

```
py -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
```

9.2 Install dependencies

Create requirements.txt with the packages used by the supplied source:

- `fastapi`
- `uvicorn[standard]`
- `python-multipart`
- `python-dotenv`
- `pypdf`
- `langchain-groq`
- `langchain-core`
- `langchain-text-splitters`
- `langchain-community`
- `sentence-transformers`
- `faiss-cpu`
- `streamlit`
- `requests`

```
python -m pip install -r requirements.txt
```

9.3 Configure the API key

Create a .env file in the location expected by api.py and add:

```
GROQ_API_KEY=your_groq_api_key_here
```

Use your real key locally. Never publish it in source control, screenshots, or this documentation.

9.4 Start the backend

Open Terminal 1 in the directory containing api.py and run:

```
uvicorn api:app --reload --host 127.0.0.1 --port 8000
```

If the backend file is in a subfolder or has a different module name, adjust the command accordingly. Check http://127.0.0.1:8000 and http://127.0.0.1:8000/docs.

9.5 Start the frontend

Open Terminal 2 in the directory containing the Streamlit frontend. If you rename main(1).py to app.py, run:

```
streamlit run app.py
```

Alternatively, run streamlit run "main(1).py" if you keep the uploaded filename.

10. Example API Requests

Ask a question

```
POST /ask\nContent-Type: application/json\n\n{"question": "What are the key findings?"}
```

Typical successful upload response

```
{"message":"PDF uploaded and processed
successfully!","filename":"example.pdf","pages_processed":10,"chunks_created":24}
```

11. Error Handling and Troubleshooting

Problem	Suggested check
Could not connect to FastAPI / connection refused	Start the backend first and confirm it is listening on 127.0.0.1:8000. Keep backend and frontend running in separate terminals.
GROQ_API_KEY is missing	Check the .env file path used by api.py, confirm the variable spelling, and restart Uvicorn after changing it.
Module import error for api:app	Run Uvicorn from the folder containing api.py using uvicorn api:app --reload. The part before the colon is the Python filename without .py; the part after it is the FastAPI variable.
No readable text found	The PDF may be scanned or image-only. The current code does not perform OCR; use a text-based PDF or add an OCR pipeline.
PDF upload rejected	Confirm the file is a valid, non-empty PDF and that its filename ends with .pdf.
Slow first upload	The sentence-transformer model may need to download on first use, and embedding/index creation can take time.
Answers miss information	The current QA flow retrieves four chunks. Results depend on PDF text extraction, chunking, and retrieval quality; the model is instructed to say when the answer is not found in context.

History disappears	History is stored in Streamlit session state, not a database. It may not persist across a new session or server restart.

12. Security, Limitations, and Future Improvements

Security considerations

- Keep .env out of GitHub and add it to .gitignore.
- Do not expose a development server directly to the public internet without authentication, rate limiting, and deployment hardening.
- Uploaded PDF content is sent as context to the configured language-model provider when asking questions or generating summaries; avoid processing confidential files unless permitted by your data-handling requirements.

Current implementation limitations

- Only one active PDF is stored in the backend at a time.
- The FAISS index and active document metadata are stored in process memory and reset when the backend process restarts.
- The history is session-based and is not stored in a database.
- Scanned PDFs are not OCR-processed.
- The frontend API URL is hard-coded for local development.
- The backend uses global state, so a multi-user production deployment could cause users to overwrite each other's active document.

Potential future improvements

- Add OCR for scanned documents.
- Persist documents and assessment history in a database or object storage.
- Use per-user document sessions and authorization.
- Add multiple-file processing and document selection.
- Add export options for answers and summaries (PDF/DOCX).
- Add automated tests for API endpoints and error cases.
- Configure production environment variables and deployment-specific frontend/backend URLs.
- Add upload size limits, request timeouts, logging, and monitoring.

13. Conclusion

The Document Analysis Agent combines a Streamlit interface with a FastAPI service and a retrieval-augmented generation workflow. It supports PDF ingestion, semantic retrieval, grounded question answering with source metadata, AI-generated summaries, and a session-based assessment history. The supplied implementation is a useful local project foundation; persistent storage, OCR, per-user isolation, and production security are important next steps before public deployment.

Appendix A. Source Files Reviewed

- api.py — FastAPI backend, PDF extraction, chunking, embeddings, FAISS, question answering, summarization, and reset.
- main(1).py — Streamlit frontend, upload workflow, feature tabs, source display, and file assessment history.

This document describes the code supplied for review. It does not claim that the application has been deployed or independently tested.